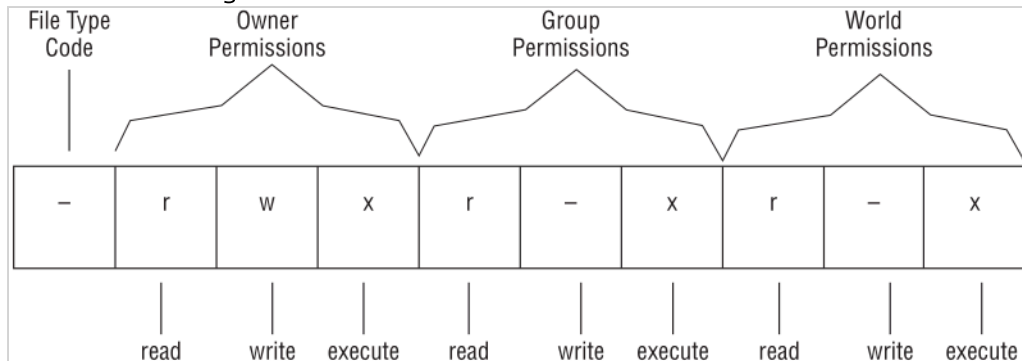


Permissions

permissions don't apply to root
 however root still needs execute bit to run a program
 but can change permissions on any file

Permission strings

10 characters long



File type

- regular file
- d directory
- l symbolic link

execute - file may be run as a program

Permission bits

1. individual permissions e.g. execute access for file owner, are often referred to as permission bits
2. linux encodes this information in binary
3. permission information can be expressed as a 9-bit number
4. the number is expressed in octal (base 8)
5. the base-8 representation of a permission string is three characters long; one for owner, group and world

octal permissions:

- 1 execute
- 2 write
- 4 read

example

```
-rwxrw-r--
```

```
111110100
```

1. group bits
 $111\ 110\ 100$
2. convert to octal
 $111\ 2^0\ 2^1 + 2^2 = 7$
 $110\ 2^1 + 2^2 = 6$
 $100\ 2^2 = 4$
3. combine octal digits
 764

Common permissions

string	octal
<code>rwxrwxrwx</code>	<code>777</code>
<code>rwx-----</code>	<code>700</code>
<code>rwxr-xr-x</code>	<code>755</code>

Special Permissions

Set User ID (SUID)

run the program with the permissions of whoever owns the file
e.g. if a file is owned by root, the program runs with root privileges

SUID is shown by a **S** in the owners execute bit
if set on a file without user execute permissions, shown by **S**
e.g. `rwSr-xr-x`

Set Group ID (SGID)

sets the group permissions
same rules as SUID
e.g. `rwxr-s-x`

Sticky Bit

set on a directory
shown as **t**
directories files can only be deleted by:

- their owners
- directories owner
- root

e.g. `rwxr-xr-t`

Access Control List (ACL)

Discretionary Access Control (DAC)

uses permissions string

Access Control List (ACL)

set per file
list of users/groups and the permissions they are given
set - `setfacl`
view - `getfacl`

Changing Mode

modify file permissions

chmod

change mode

`chmod [options] [mode] filename`

`-R --recursive` changes all files in directory tree

example

`chmod 644 file.dat`

Setting Special Permissions

add a number before the three octal digits

setting the first digit as 0 removes all special permissions

bit permissions:

1 sticky bit permission

2 SGID permission

4 SUID permission

example

```
chmod 6750 program.sh
```

SGID = 2

SUID = 4

2+4=6

Symbolic Mode

codes for permissions

Permission set code	Meaning	Change type code	Meaning	Permission to modify code	Meaning
u	Owner	+	Add	r	Read
g	Group	-	Remove	w	Write
o	Other	=	Set equal to	x	Execute
a	All			X	Execute only if the file is a directory or already has execute permission
				s	SUID or SGID
				t	Sticky bit
				u	Existing owner's permissions
				g	Existing group permissions
				o	Existing other permissions

example

```
chmod a+x bigprogram
```

```
chmod g=u bigprogram# Default Permissions
```

When creating a file, it has default permissions

owner is whoever created the file

group is the users primary group

default permissions are configurable

umask

user mask

takes an octal value that represents bits to be removed from either:

777 directories (default)

666 files (default)

examples

umask	created files	created directories
000	666 (rw-rw-rw-)	777 (rwxrwxrwx)
022	644 (rw-r--r--)	755 (rwxr-xr-x)
077	600 (rw-----)	700 (rwx-----)

default umask

```
james@laptop:~$ umask
0002
james@laptop:~$ umask -S
u=rwx,g=rwx,o=rx
james@laptop:~$
```

the first digit is the octal code for SUID, SGID and the sticky bit

To change umask

1. `umask 0022`
changes to 0022
2. `umask u=rwx,g-rx,o=rx`
equivalent to method 1

Default Group

users can change their default group

newgrp

new group

the user using this command must be a member of that group

example `newgrp skygroup` # File Attributes

you can change attributes for most file systems using `chattr`

chattr

`chattr +/-[option] file`

options

A	no access time updates don't updated access timestamp when accessing a file
i	immutable don't updated access timestamp when accessing a file

example

`chattr +i important.txt`